`tests/dynamic/analyzers/no_leak_analyzer.sh` — operator guide

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Status:** Active self-validated analyzer (§11.4.107(10)). Signal 1 of 6 for the dynamic data plane — fail-closed kill-switch / no-leak.

> Companion (§11.4.18) to the in-source documentation block at the top of the script.

Overview

Signal 1: **zero target packets on the real uplink while the tunnel is DOWN** (§11.4.69 `network_connectivity`; design §10/§13 `no_leak`). Given a `tcpdump` capture taken **on the real uplink** while the target VPN tunnel is DOWN, it PASSes iff **zero** packets escaped to the target. It delegates the zero-packet decision to the committed, self-tested `evidence.sh:assert_no_leak` (`tcpdump "N packets captured"`, a `/proc/net/dev` tx-packet delta, or a raw `" IP "` line count) and adds an optional dynamic-mode target-host filter.

Prerequisites

- Source library `tests/dynamic/lib/analyzer_common.sh` + the committed `tests/lib/evidence.sh` (it delegates to `assert_no_leak`).
- `grep`, `awk`, POSIX `sh`.
- A real uplink capture taken during the tunnel-DOWN window (live P10 input); the self-test feeds bundled fixtures with no network.

Usage examples

```
# Analyze a real-uplink capture (PASS iff 0 target packets during
        DOWN window):
tests/dynamic/analyzers/no_leak_analyzer.sh analyze <capture-
        file> [target-host-or-ip]
```

```
# Self-validate (golden-good PASS + golden-bad FAIL) — the default
        action:
tests/dynamic/analyzers/no_leak_analyzer.sh --selftest
```

Edge cases

- **Capture file missing** → `FAIL` (`capture file missing`), rc 1.
- **`evidence.sh` not found** → `FAIL` (`cannot delegate`), never a bluff PASS.
- **Target supplied on a raw text capture** (no `packets captured` footer, not a `/proc/net/dev` delta) → counts only `" IP …<target>"` lines, so unrelated host chatter on a shared uplink neither masks nor fakes a leak.
- **Tunnel UP** — testing for leaks while the tunnel is up proves nothing; this analyzer asserts fail-closed during the **DOWN window only** (§11.4.107 honest window).

§11.4.115 RED_MODE polarity

The dynamic suites that consume this analyzer run the §11.4.115 polarity (RED reproduces a leak on the pre-fix/broken stack; GREEN guards zero-leak). The analyzer itself is the oracle both polarities cite as captured evidence.

Internal behaviour

- `#!/usr/bin/env bash`, POSIX-clean (`sh -n` + `bash -n`, §11.4.67).
- Dispatch: `analyze` → `analyze_no_leak`; `--selftest`/`selftest`/empty → `_selftest_no_leak`.
- golden-good: a real-uplink capture during the DOWN window with 0 target packets. golden-bad: the SAME capture WITH a leaked target packet → MUST FAIL.
- Self-test covers tcpdump-footer, `/proc/net/dev` delta, and target-filtered raw forms, plus the missing-file negative.

Related

- `tests/lib/evidence.sh` (`assert_no_leak`) — the delegated oracle.
- `tests/dynamic/lib/analyzer_common.sh` — sourced base.
- `tests/dynamic/suites/chaos_suite.sh` — consumes this analyzer (C1 no-leak).
- Fixtures: `tests/dynamic/analyzers/fixtures/no_leak/`.
- Constitution §11.4.69 / §11.4.107 / §11.4.115; design §10/§13; research §5.

Last verified

2026-07-01 — self-test PASS (golden-good PASS + golden-bad FAIL across tcpdump, procdev-delta, and target-filter forms); `sh -n` + `bash -n` parse-clean. Live real-uplink capture is exercised in **P10**.